Ibex / Nangate45 時鐘樹合成（CTS）策略研究 — 工作報告

從頭到尾做了什麼、跑了什麼、寫了什麼、輸出了什麼。期間：2026-11-03 07:50 ~ 2026-11-04（UTC）。
Repo：`https://github.com/chengsien/ibex-cts-study`（main）。

本報告中所有數字皆由 `scripts/summarize.py` 直接讀取各 run 的實際報告產生，沒有手填或估計值。設計為開源 Ibex RISC-V 核心（lowRISC），製程為開源 FreePDK45 / Nangate Open Cell Library，非 foundry 製程，單一 typical corner。

ORFS run 總數	最短收斂週期	預設 → 最佳 WNS（2.2 ns）	Git commits
24 10 個從頭完整流程，14 個從 CTS 分支	2.25 ns 壓力週期 2.2 ns（差 14 ps）	−0.0143 → +0.0235 ns 兩輪實體 useful skew	11 全部已 push

1. 結論摘要

- 環境：Docker Hub 拉 image 被 429 擋住，改從 Docker Hub 直接下載官方 image 的 26 個 layer（SHA256 驗證）自行組回，得到與 clone 的 ORFS commit 完全一致的 OpenROAD / Yosys(slang) / KLayout。預設 ibex 完整流程約 8.6 分鐘、0 DRC。
- Phase 1：掃描 1.8–2.6 ns（7 點）。route 後（OpenRCX）能收斂的最短週期為 2.25 ns（WNS +0.013 ns）；2.2 ns（ORFS 預設）差 14 ps 未收斂，作為壓力週期。
- 可重現性：流程完全確定性（R0a = R0b = RA，逐位元一致），重跑雜訊為 0；但只改 global routing 容量的雜訊探針就能讓 WNS 變動 +0.4～+20.6 ps，所以比較時把 slack 拆成「時鐘部分」（探針中 ≤0.9 ps，可信）與「資料路徑部分」（探針中最多 27.7 ps，屬雜訊）。
- CTS 選項查證：此版 ORFS 預設已開 sink clustering、OpenROAD 預設已套 NDR `half`；`-balance_levels` 已廢棄；沒有內建 useful skew。
- CTS 參數效果（RA 的 31 個失敗 endpoint 的時鐘部分平均改善）：關 clustering +18.9 ps（clock power +2.1%）、cluster size 4 +14.6 ps（clock power +26%）、NDR full +13.9 ps（幾乎無代價）；cluster size 20 與 NDR root_only 完全無效果。
- Useful skew：STA 模擬 +26.7 ps；實際插 15 顆 CLKBUF_X1 +36.2 ps、WNS +0.0185 ns；第二輪（再 21 顆）+45.7 ps、WNS +0.0235 ns，報酬遞減明顯。全程 0 個 hold 修復 cell。
- 未完成：把 launch flop 時鐘提早（RE）的實驗已規劃但尚未執行（在你要求本報告時暫停，沒有任何 run 或檔案改動）。

2. 工作時間線

時間（UTC）	階段	內容
11-03 07:51	Step 0	環境探查：容器無 repo、Docker daemon 未啟動；clone ORFS；詢問 push 目標 repo
07:53–08:05	Step 0	嘗試安裝：docker pull（429）、mirror.gcr.io（429）、Precision Innovations .deb（GitHub API 被擋）、conda litex-hub（版本停在 2024-02，無 slang）→ 改從 layer blob 重建 image
08:06–08:15	Step 0	預設 ibex 完整流程 step0_default（約 8.6 分鐘），寫 clock_report.tcl / summarize.py，修兩個 bug
08:20–09:09	Phase 1	週期掃描 1.8/2.0/2.2/2.3/2.4/2.6 + 補 2.25 ns；同時查 CTS 文件、寫 RD 腳本（修 arrival 屬性 bug）
08:33 之後	—	你建立 repo、授權 GitHub App 後首次 push（之前的 3 個 commit 一起推上）

時間（UTC）	階段	內容
09:10–10:26	Phase 2	R0a/R0b、7 個 CTS 變體、補 RC_root、雜訊探針（第一版無效後改用 PRE_GLOBAL_ROUTE hook）、slack 分解、findings、README
11-04 ~ 02:05	RD_phys	useful skew 實體化：insert_buffer 插延遲 buffer（CLKBUF_X1 / BUF_X1），改用 per-endpoint 分解並更正 RB_s4 的結論
~07:34	Round 2	從 route 後時序選第二輪目標：只處理 rf_reg[444] 與處理整個 <30 ps 層兩種
之後	RE（暫停）	規劃 launch flop 時鐘提早；本報告製作

3. 環境建置（Step 0）

#	方法	結果
1	`docker pull openroad/orfs:26Q4-260-gc9ac73efc`（ORFS 官方預編譯 image）	失敗：Docker Hub 429，共用出口 IP 的匿名額度 `ratelimit-remaining: 0`；mirror.gcr.io 同樣 429
2	Precision Innovations 預編譯 .deb（GitHub release）	失敗：session 網路政策只允許已掛載 repo 的 GitHub API（403）
3	conda litex-hub channel	可連線但最新版是 2024-02-23（OpenROAD 2.0_12381、Yosys 0.38），無 yosys-slang，與 2026 的 ORFS 不相容 → 放棄
4	從 layer blob 重建官方 image（`scripts/fetch_orfs_image.sh`）	成功：manifest 才受限、blob 不受限；26 個 blob（1.56 GB）40 秒下載完並逐一 SHA256 驗證，套用 OCI whiteout 後 `docker import`

工具	版本
ORFS	commit `c9ac73efc603…`（2026-11-03）；image 內 flow/ 與 clone 逐檔相同
OpenROAD	ORFS 子模組 `98c6045c72b7`（2026-10-27）；`-version` 顯示 unknown（CI 建置無 git 資訊）
OpenSTA	`384b539085c5`
Yosys	0.68+post（內建 slang 前端）
KLayout	0.30.12
主機	Ubuntu 24.04、4 vCPU、15 GB RAM；image 為 Ubuntu 26.04

設計與約束（Phase 1.1–1.2）

項目	值
時鐘埠 / 時鐘名	`clk_i` / `core_clock`（另有 I/O 用的虛擬時鐘 `vclk_core_clock`）
預設週期 / 時間單位	2.2 ns / ns（Liberty time_unit 1ns）
RTL	lowRISC Ibex，commit `77d80100`（ORFS 精簡版），SystemVerilog 經 yosys-slang 讀入
暫存器堆	`RegFileFF`（flip-flop 實作，31×32 bit）
Core utilization	50%（`CORE_UTILIZATION ?= 50`）
Lib corner	只有 `NangateOpenCellLibrary_typical.lib`（typical, 1.10 V, 25 °C）；其餘為 fakeram（ibex 未用）
規模	1,938 個時鐘 sink（1,937 DFF + 1 個 clock-gate latch）、15,091 個 cell（不含 fill）、core 約 237×237 µm

4. 跑了什麼：全部 24 個 ORFS run

每個 run 都走完 synth→floorplan→place→CTS→global route→detailed route→fill→OpenRCX 寄生萃取→STA→GDS。「分支自 R0a」表示複製 R0a 的 synthesis/floorplan/placement 結果（`3_place.odb` 逐位元相同）後從 CTS 重跑，確保唯一變數是 CTS 設定。WNS/TNS/hold 為 route 後 OpenRCX SPEF 的結果。牆鐘時間是兩個 run 並行時量測。

#	Run	類別	目的	設定	牆鐘 (s)	WNS (ns)	TNS (ns)	hold WNS	DRC
1	step0_default	Step 0	ORFS 預設 ibex 完整流程（環境可行性檢查）	2.2 ns	515	−0.0143	−0.360	+0.0855	0
2	P1p8	Phase 1	週期掃描 1.8 ns	1.8 ns	947	−0.2157	−216.611	+0.0918	0
3	P2p0	Phase 1	週期掃描 2.0 ns	2 ns	778	−0.0199	−2.738	+0.0866	0
4	P2p2	Phase 1	週期掃描 2.2 ns（= ORFS 預設週期）	2.2 ns	668	−0.0143	−0.360	+0.0855	0
5	P2p25	Phase 1	週期掃描 2.25 ns（補點，定位收斂邊界）	2.25 ns	574	+0.0125	+0.000	+0.0843	0
6	P2p3	Phase 1	週期掃描 2.3 ns	2.3 ns	679	+0.0227	+0.000	+0.0850	0
7	P2p4	Phase 1	週期掃描 2.4 ns	2.4 ns	674	+0.0177	+0.000	+0.0891	0
8	P2p6	Phase 1	週期掃描 2.6 ns	2.6 ns	715	+0.1180	+0.000	+0.0848	0
9	R0a	Phase 2	預設設定，從頭完整跑（基準 #1，也是所有分支的 placement 來源）	2.2 ns	776	−0.0143	−0.360	+0.0855	0
10	R0b	Phase 2	預設設定，從頭完整跑（基準 #2，可重現性）	2.2 ns	775	−0.0143	−0.360	+0.0855	0
11	RA	Phase 2	對照組：預設 TritonCTS，從 R0a placement 重跑 CTS→finish	2.2 ns；分支自 R0a	619	−0.0143	−0.360	+0.0855	0
12	RB_nocl	Phase 2	關閉 sink clustering	2.2 ns；分支自 R0a；CTS_ARGS= `-repair_clock_nets`	634	+0.0043	+0.000	+0.0853	0
13	RB_s4	Phase 2	sink cluster size = 4	2.2 ns；分支自 R0a；CTS_CLUSTER_SIZE=4	602	+0.0102	+0.000	+0.0848	0
14	RB_s20	Phase 2	sink cluster size = 20	2.2 ns；分支自 R0a；CTS_CLUSTER_SIZE=20	580	−0.0143	−0.360	+0.0855	0
15	RC_none	Phase 2	NDR = none	2.2 ns；分支自 R0a；CTS_ARGS= `-sink_clustering_enable -repair_clock_nets -apply_ndr none`	571	−0.0090	−0.213	+0.0855	0
16	RC_root	Phase 2	NDR = root_only	2.2 ns；分支自 R0a；CTS_ARGS= `-sink_clustering_enable -repair_clock_nets -apply_ndr root_only`	656	−0.0143	−0.360	+0.0855	0
17	RC_full	Phase 2	NDR = full	2.2 ns；分支自 R0a；CTS_ARGS= `-sink_clustering_enable -repair_clock_nets -apply_ndr full`	601	+0.0164	+0.000	+0.0855	0

#	Run	類別	目的	設定	牆鐘 (s)	WNS (ns)	TNS (ns)	hold WNS	DRC
18	RD	Phase 2	useful skew 模擬：set_clock_latency 覆寫 62 個暫存器（只在 STA）	2.2 ns；分支自 R0a；POST_CTS_TCL=`rd_useful_skew.tcl`	392	+0.0139	+0.000	+0.0855	0
19	N_gr48	雜訊探針	只改 global routing 容量 0.50→0.48（時鐘樹不變）	2.2 ns；分支自 R0a；PRE_GLOBAL_ROUTE_TCL=`noise_grt_adj_48.tcl`	576	−0.0139	−0.347	+0.0855	0
20	N_gr52	雜訊探針	只改 global routing 容量 0.50→0.52（時鐘樹不變）	2.2 ns；分支自 R0a；PRE_GLOBAL_ROUTE_TCL=`noise_grt_adj_52.tcl`	401	+0.0063	+0.000	+0.0855	0
21	RD_phys	Useful skew	實體插 15 顆 CLKBUF_X1 延遲 62 個 capture 暫存器	2.2 ns；分支自 R0a；POST_CTS_TCL=`rd_phys_useful_skew.tcl`	642	+0.0185	+0.000	+0.0856	0
22	RD_phys_b1	Useful skew	同上，改用 BUF_X1	2.2 ns；分支自 R0a；POST_CTS_TCL=`rd_phys_useful_skew_bufx1.tcl`	642	+0.0146	+0.000	+0.0856	0
23	RD_phys2_444	Useful skew	第二輪：只處理 rf_reg[444]（+1 顆 buffer）	2.2 ns；分支自 R0a；POST_CTS_TCL=`rd_phys2_444.tcl`	701	+0.0167	+0.000	+0.0854	0
24	RD_phys2	Useful skew	第二輪：處理 route 後 <30 ps 的 57 個暫存器（+21 顆 buffer）	2.2 ns；分支自 R0a；POST_CTS_TCL=`rd_phys2_T30.tcl`	701	+0.0235	+0.000	+0.0854	0

step0_default 的牆鐘時間是由檔案時間戳推得（該 run 進行中我修改了 run.sh，導致包裝腳本最後一行記錄失敗；流程本身完整）。

每個 run 之後另外執行的分析（以 `make run RUN_SCRIPT=…` 在同一 image 中）

- `post_reports.sh` → `clock_report.tcl`：CTS 與 final 兩個資料庫各一次（全部 24 個 run）
- `worst_paths.tcl`、`endpoint_slack.tcl`、`dump_ck_arrivals.tcl`：Phase 2 的 16 個 run
- `rd2_select.tcl`：在 RD_phys 的 route 後資料上跑兩次（T30 + stack、只選 rf_reg[444]）
- `ndr_check.tcl`：RA、RC_none、RC_root、RC_full、RB_nocl、RB_s4
- 獨立測試（結果只放 `.scratch/`，不進 repo）：RD/RD_phys/RD_phys2 hook 在 R0a CTS 資料庫上的試跑、CK arrival 偵錯、launch/hold 檢查

5. 寫了什麼

腳本（scripts/）

檔案	行數	用途
`scripts/clock_report.tcl`	163	時鐘指標：clock buffer 數/種類/面積、dummy load、delay buffer、insertion delay min/max/mean、skew、WNS/TNS、功耗分項、時鐘繞線長度
`scripts/dump_ck_arrivals.tcl`	17	route 後每個暫存器 CK 腳的時鐘到達時間與驅動 cell
`scripts/endpoint_slack.tcl`	13	route 後每個 slack <100 ps 的 endpoint：slack、capture 延遲、startpoint
`scripts/fetch_orfs_image.sh`	51	Docker Hub 429 時，從 layer blob 重建 ORFS image（SHA256 驗證、處理 OCI whiteout、docker import）
`scripts/ndr_check.tcl`	14	數出資料庫中帶 NDR 的時鐘線數
`scripts/noise_grt_adj_48.tcl`	7	雜訊探針 hook：global routing 容量 0.48

檔案	行數	用途
`scripts/noise_grt_adj_52.tcl`	7	雜訊探針 hook：global routing 容量 0.52
`scripts/phase2.sh`	62	Phase 2 全矩陣驅動（R0a/R0b → 各 CTS 變體 → useful skew 兩輪 → 分析）
`scripts/post_reports.sh`	19	對一個 run 在 CTS 與 final 資料庫上執行 clock_report.tcl
`scripts/rd2_select.tcl`	92	第二輪 useful skew 目標選擇（用 route 後 SPEF 時序；可疊第二級）
`scripts/rd_phys2_444.tcl`	5	RD_phys2_444 包裝（只處理 rf_reg[444]）
`scripts/rd_phys2_T30.tcl`	5	RD_phys2 包裝（<30 ps 全層）
`scripts/rd_phys2_useful_skew.tcl`	137	第一輪＋第二輪 useful skew 插入
`scripts/rd_phys_useful_skew.tcl`	175	RD_phys：同樣選擇規則，用 insert_buffer 實際插入延遲 buffer，量測實際延遲並有 guard 移除機制
`scripts/rd_phys_useful_skew_bufx1.tcl`	5	RD_phys_b1 包裝（改用 BUF_X1）
`scripts/rd_useful_skew.tcl`	93	RD：以 set_clock_latency 在 STA 中模擬 useful skew
`scripts/run.sh`	118	單次 ORFS run 包裝：週期、utilization、CTS 參數、cluster 大小/直徑、POST_CTS hook、從既有 placement 分支（-b），輸出到 runs/<name>/ 並記錄 run_config.txt
`scripts/summarize.py`	388	從所有 run 報告擷取指標 → results/*.csv 與圖；含 per-endpoint 時鐘/資料路徑分解
`scripts/sweep_period.sh`	11	Phase 1 週期掃描驅動（兩個 run 並行）
`scripts/worst_paths.tcl`	29	route 後前 10 條最差路徑，拆成 launch/capture 時鐘延遲與資料延遲

文件

檔案	行數	內容
`README.md`	162	目標、流程、重現方式、結果摘要表；註明 Ibex 來源與開源 PDK（非 foundry 製程）
`fetch_inputs.md`	43	所有輸入（ORFS、image、RTL、PDK）的取得方式與版本，repo 內不含 PDK/庫檔
`docs/setup-notes.md`	123	Step 0 安裝嘗試與工具版本、Phase 1 的 config/SDC/lib corner 紀錄、run.sh 說明
`docs/experiment-log.md`	600	E0–E8 每個實驗：問題 → 做法 → 結果表 → 發現 → 學到（含未解決問題與更正）
`docs/findings.md`	197	四個問題的回答與總結排名
`constraints/ibex_constraint.sdc`	19	ORFS ibex SDC 的逐位元組副本（每個 run 只替換 clk_period）

6. 輸出了什麼

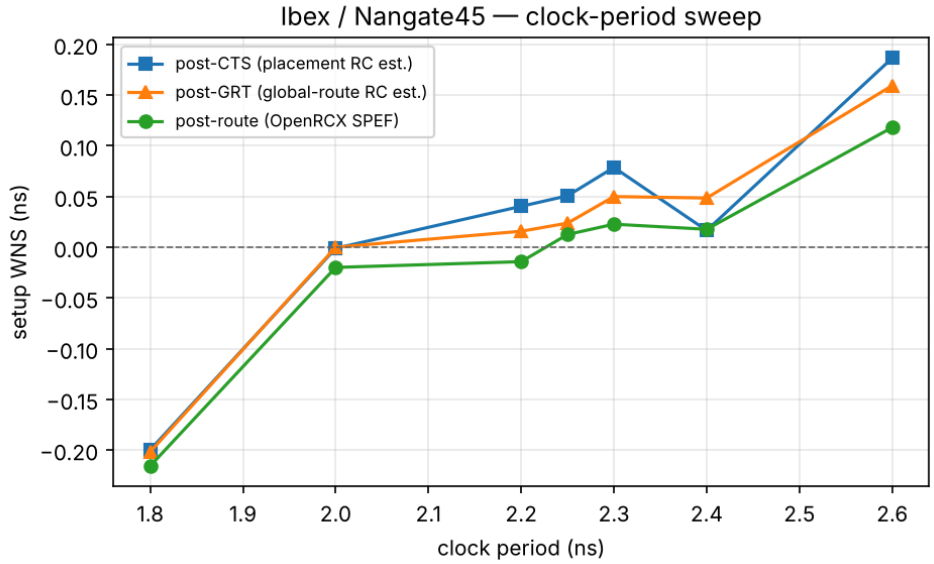
輸出	內容
`results/period_sweep.csv` / `period_sweep_wns.png`	Phase 1：7 個週期的 CTS 後 / GRT 後 / route 後 WNS、TNS、面積、功耗等
`results/cts_comparison.csv` / `cts_comparison_full.csv`	Phase 2：16 個 run 的精簡欄位 / 全部欄位（約 150 欄）
`results/cts_comparison.png`	六項指標（insertion delay、skew、clock buffer、WNS、TNS、clock power）
`results/cts_slack_decomposition.png`	31 個失敗 endpoint 的 slack 變化拆成時鐘 / 資料路徑兩部分，含雜訊帶
`results/figures/*.webp`	baseline 的時鐘樹、繞線截圖（ORFS 產生）
`runs/<run>/`（進 git 的部分）	每個 run 的 logs、reports、run_config.txt、constraint.sdc、分析輸出（CSV/kv/rpt）；共 1631 個檔案

輸出	內容
`runs/<run>/`（不進 git）	ODB、DEF、GDS、SPEF、網表、webp 等大檔（.gitignore），本機約 7.7 GB
Git	11 個 commit、repo 共 1667 個追蹤檔案，全部 push 到 main

7. 主要結果

7.1 Phase 1：週期掃描（route 後，OpenRCX）

週期 (ns)	CTS 後 WNS	GRT 後 WNS	route 後 WNS	TNS	失敗 endpoint	hold WNS	cell 面積 μm²	總功耗 mW	結果
1.8	−0.200	−0.202	−0.216	−216.61	1580	+0.092	32,643	16.04	未收斂
2	−0.001	−0.000	−0.020	−2.74	269	+0.087	30,145	14.17	未收斂
2.2	+0.040	+0.016	−0.014	−0.36	31	+0.085	29,018	12.71	未收斂
2.25	+0.051	+0.024	+0.013	+0.00	0	+0.084	28,958	12.38	收斂
2.3	+0.079	+0.050	+0.023	+0.00	0	+0.085	28,898	12.10	收斂
2.4	+0.016	+0.048	+0.018	+0.00	0	+0.089	28,733	11.63	收斂
2.6	+0.186	+0.159	+0.118	+0.00	0	+0.085	28,604	10.86	收斂



發現：2.0–2.2 ns 的失敗主要來自 global-route RC 估計與 OpenRCX 萃取之間的差距（repair_timing 把 GRT 階段推到約 0，萃取後再掉 20–30 ps）；曲線不單調（每個週期的合成與擺放都不同）；時鐘樹在各週期幾乎一樣（316–321 顆 CLKBUF_X3）。

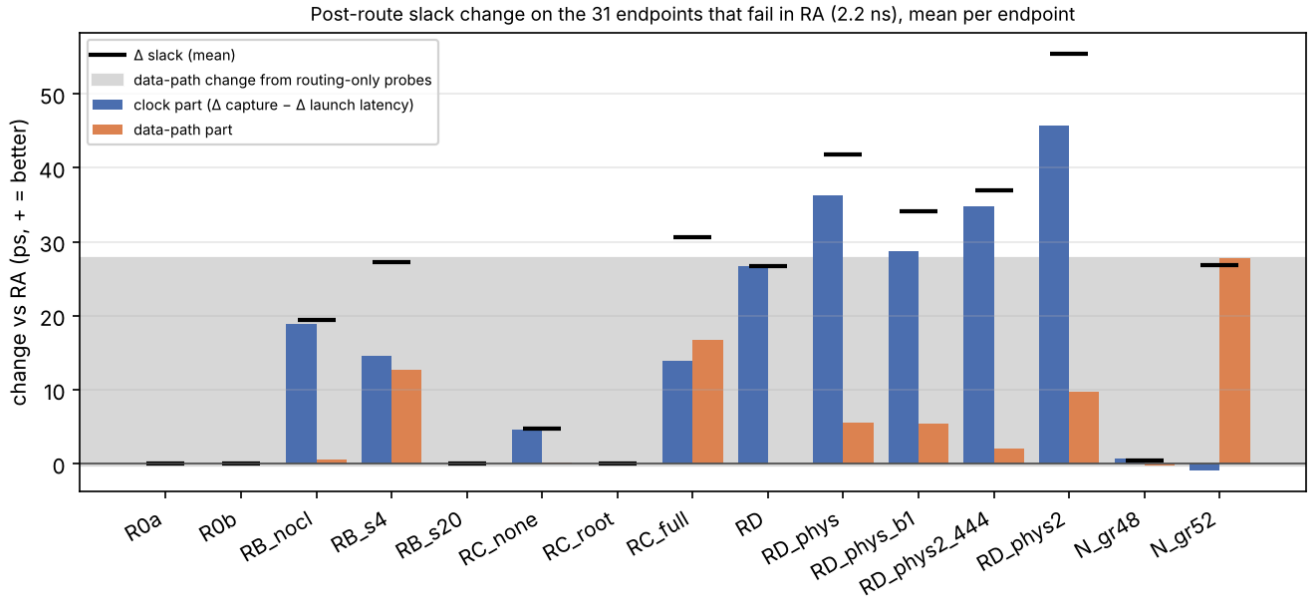
7.2 Phase 2：CTS 策略比較（2.2 ns，同一 placement）

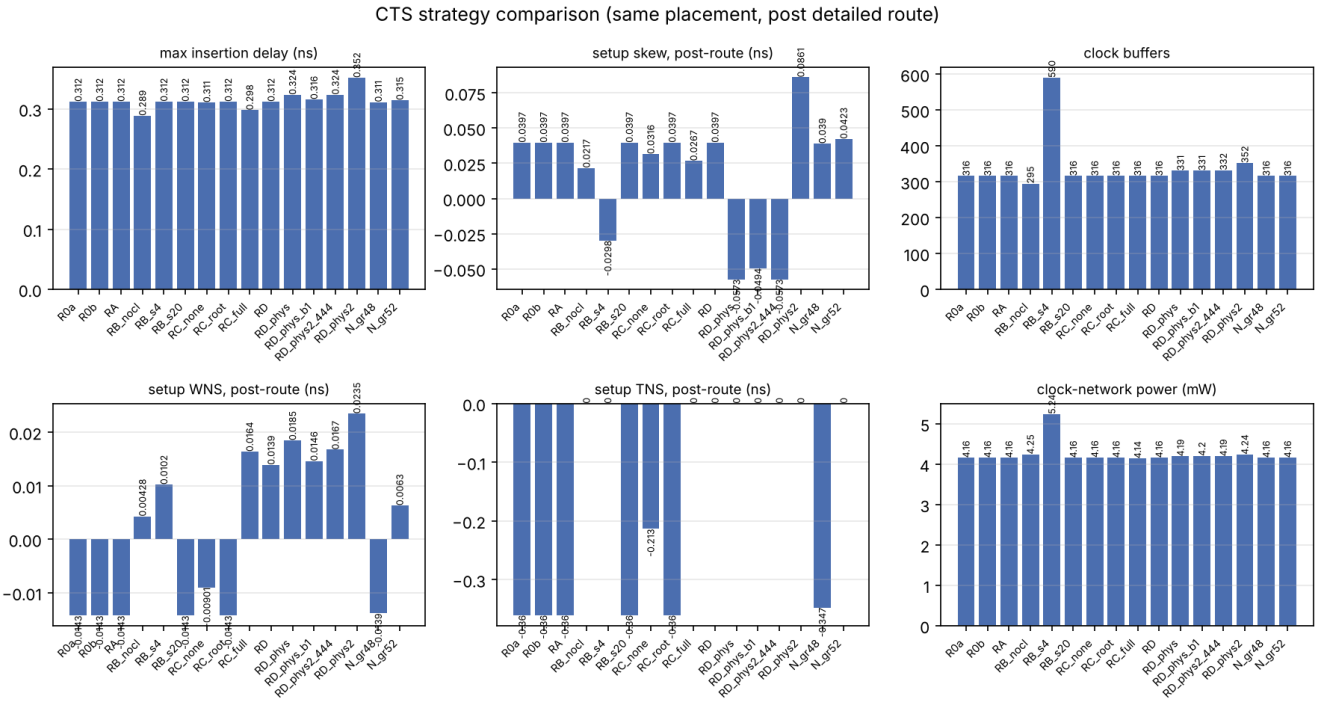
RA 在 2.2 ns 有 31 個失敗 endpoint，全部是暫存器堆的 flop，且全部來自同一個起點 `if_stage_i.instr_rdata_id_o[20]`。下表 Δslack 為這 31 個 endpoint 相對 RA 的平均 slack 變化（ps，正值較好），逐 endpoint 拆成時鐘部分（Δcapture − Δlaunch 延遲）與資料路徑部分。

Run	改了什麼	Δslack	時鐘部分	資料部分	WNS (ns)	TNS (ns)	最大 insertion delay	clock buffer	hold WNS / hold 修復 cell	clock / 總功耗 (mW)
RA	對照組：預設 TritonCTS，從 R0a placement 重跑 CTS→finish	+0.0	+0.0	+0.0	−0.0143	−0.360	0.312	316	+0.0855 / 0	4.158 / 12.706
RB_nocl	關閉 sink clustering	+19.4	+18.9	+0.6	+0.0043	+0.000	0.289	295	+0.0853 / 0	4.245 / 12.795

Run	改了什麼	Δslack	時鐘部分	資料部分	WNS (ns)	TNS (ns)	最大 insertion delay	clock buffer	hold WNS / hold 修復 cell	clock / 總功耗 (mW)
RB_s4	sink cluster size = 4	+27.3	+14.6	+12.7	+0.0102	+0.000	0.312	590	+0.0848 / 0	5.239 / 13.782
RB_s20	sink cluster size = 20	+0.0	+0.0	+0.0	−0.0143	−0.360	0.312	316	+0.0855 / 0	4.158 / 12.706
RC_none	NDR = none	+4.7	+4.6	+0.1	−0.0090	−0.213	0.311	316	+0.0855 / 0	4.162 / 12.710
RC_root	NDR = root_only	+0.0	+0.0	+0.0	−0.0143	−0.360	0.312	316	+0.0855 / 0	4.158 / 12.706
RC_full	NDR = full	+30.6	+13.9	+16.7	+0.0164	+0.000	0.298	316	+0.0855 / 0	4.140 / 12.688
RD	useful skew 模擬：set_clock_latency 覆寫 62 個暫存器（只在 STA）	+26.6	+26.7	−0.0	+0.0139	+0.000	0.312	316	+0.0855 / 0	4.158 / 12.706
RD_phys	實體插 15 顆 CLKBUF_X1 延遲 62 個 capture 暫存器	+41.8	+36.2	+5.6	+0.0185	+0.000	0.324	331	+0.0856 / 0	4.193 / 12.742
RD_phys_b1	同上，改用 BUF_X1	+34.1	+28.7	+5.5	+0.0146	+0.000	0.316	331	+0.0856 / 0	4.201 / 12.748
RD_phys2_444	第二輪：只處理 rf_reg[444]（+1 顆 buffer）	+36.9	+34.8	+2.1	+0.0167	+0.000	0.324	332	+0.0854 / 0	4.195 / 12.743
RD_phys2	第二輪：處理 route 後 <30 ps 的 57 個暫存器（+21 顆 buffer）	+55.4	+45.7	+9.7	+0.0235	+0.000	0.352	352	+0.0854 / 0	4.241 / 12.789
N_gr48	只改 global routing 容量 0.50→0.48（時鐘樹不變）	+0.4	+0.7	−0.3	−0.0139	−0.347	0.311	316	+0.0855 / 0	4.158 / 12.706
N_gr52	只改 global routing 容量 0.50→0.52（時鐘樹不變）	+26.8	−0.9	+27.7	+0.0063	+0.000	0.315	316	+0.0855 / 0	4.158 / 12.702

R0a、R0b 與 RA 在所有指標上完全相同。RB_s20 與 RA、RC_root 與 RA 的 CTS 資料庫逐位元相同。資料部分在雜訊探針中最多 +27.7 ps，因此不計入 CTS 效果；時鐘部分在探針中 ≤0.9 ps。所有 run：DRC 0、GRT overflow 0。





7.3 Useful skew 的演進

Run	延遲 buffer	WNS (ns)	失敗 endpoint	<20 ps endpoint	最差 endpoint	clock power (mW)	cell 面積 μm²	最大 insertion delay
RA	–	−0.0143	31	56	rf_reg[991]	4.158	29,017.9	0.312
RD	0（SDC 覆寫）	+0.0139	0	31	rf_reg[255]	4.158	29,017.9	0.312
RD_phys	15	+0.0185	0	3	rf_reg[444]	4.193	29,029.9	0.324
RD_phys2_444	16	+0.0167	0	6	rf_reg[787]	4.195	29,030.7	0.324
RD_phys2	36	+0.0235	0	0	rf_reg[863]	4.241	29,046.7	0.352

- RD（STA 模擬）：62 個暫存器 set_clock_latency 覆寫，平均目標 +13.3 ps。
- RD_phys：同樣 62 個暫存器分成 15 組各接一顆 CLKBUF_X1；route 後實際延遲 +27～+41 ps（中位數 36），比目標大約 3 倍，但這些 flop 的 launch 端仍有 ≥0.14 ns 餘裕，所以反而更好。其他 1,876 個 flop 幾乎不變（−7～

+1 ps）。

- 第二輪只處理 rf_reg[444]：它自己 +18.5 → +47.1 ps，但 WNS 反而下降 1.7 ps（re-route 雜訊大於可得的 0.3 ps）。
- 第二輪處理整個 <30 ps 層（57 個 flop、21 顆 buffer，其中 9 個疊第二級）：WNS +23.5 ps，比第一輪只多 5 ps；下一個瓶頸是其他 instruction-register 位元的路徑與 108 個 30–40 ps 的 endpoint。
- 代價（RD_phys2 相對 RA）：36 顆 buffer、clock power +2.0%、總功耗 +0.65%、面積 +0.1%；被延遲 flop 的 launch 端 slack 0.143→0.126 ns、hold 0.401→0.338 ns；hold 修復 cell 0。

8. 遇到的問題、錯誤與更正（如實記錄）

問題	處理方式	狀態
Docker Hub 429	改從 layer blob 重建 image（見第 3 節）	已解決
工作目錄沒有 repo、無法 push	請你建立 chengsien/ibex-cts-study 並授權 Claude GitHub App；之前的 commit 先留本機，授權後一次 push	已解決
step0 時在 run 進行中修改 run.sh	bash 邊讀邊執行，最後的記錄行壞掉；流程本身完整，執行時間改由檔案時間戳推算；之後不再在 run 中改腳本	已記錄
clock_report.tcl：WNS 讀成 0	OpenSTA 內部單位是秒，改用 sta::format_time	已修正，並與 ORFS metrics 比對一致
clock_report.tcl：時鐘繞線長度 API 錯誤	改用 report_wire_length -detailed_route 並解析輸出	已修正
RD 第一次測試：CK arrival 讀到 1.745 ns（應為 0.291）	呼叫 find_timing_paths 之後 pin 屬性 arrival_max_rise 會變；改為先讀所有 CK arrival 再列舉路徑，並加合理範圍檢查。根本原因未查明	已繞過；未回報上游
path 物件在下一次查詢後失效 → OpenROAD segfault	每次查詢後立即讀出 slack	已繞過
OpenSTA 與 OpenDB 對 bus 中括號跳脫不同（findInst 回傳 NULL）	改用 sta::sta_to_db_inst 轉換	已修正
第一版雜訊探針（FASTROUTE_TCL）無效	ORFS 在 floorplan 階段就把設定寫入資料庫，從 CTS 分支的 run 讀不到；發現 congestion 表與 RA 完全一樣後改用 PRE_GLOBAL_ROUTE_TCL hook，並以 log 確認生效	已更正、無效的 run 已刪除
pkill -f 誤殺自己的 shell（兩次）	pattern 同時出現在自己的命令列；改用 PID 精確停止	已記錄
「只比較最差路徑」的分解在 RD_phys 中誤導（最差 endpoint 換人）	改為對 RA 的 31 個失敗 endpoint 逐一分解取平均	已改方法
更正先前結論：曾寫 RB_s4 的改善「不是時鐘效果」	用新方法後，RB_s4 的 +27.3 ps 中有 +14.6 ps 是時鐘部分；已更正 experiment-log E3、findings、README	已更正
容器重啟（Docker daemon 停止）	重新啟動 dockerd，image 與 runs/ 都還在	已處理

9. 限制與未完成事項

- RE：把 launch flop 時鐘提早，已規劃但尚未執行。做法：把 `if_stage_i.instr_rdata_id_o[*]` 的 CK 從 leaf net 接到上一級 net，少一級 buffer，並對被它 capture 的路徑（setup）與它 launch 的路徑（hold）做保護。
- 平台只有 typical corner，hold 沒有在 fast corner 驗證；沒有 OCV/derate，也沒有 crosstalk（SI）分析，所以 NDR 的主要好處看不到。
- 雜訊探針只有 2 個樣本，是雜訊下限而非分佈。
- RD 使用的 OpenSTA pin arrival 屬性異常，根本原因未追查。
- useful skew 的延遲步距粗（一顆 CLKBUF_X1 約 36 ps），無法細調；第二輪之後報酬遞減。
- 「設計大 10 倍」的結論是推論，不是實測。

附錄：Git commit 紀錄

Commit	時間（UTC）	訊息
`3cb0acc`	11-03 08:07	Scaffold: run wrapper, image fetch script, pinned inputs doc
`c6a93da`	11-03 08:24	Step 0: ORFS image rebuilt from verified layer blobs; baseline ibex flow passes
`5fecef0`	11-03 08:33	Add RD useful-skew emulation hook (with arrival-sampling fix), NDR check, sweep driver
`145fb3d`	11-03 08:37	Phase 2 prep: CTS option verification (E0) and matrix driver
`1bee470`	11-03 09:09	Phase 1: clock-period sweep 1.8-2.6 ns; stress period = 2.2 ns
`a6b1c91`	11-03 09:20	README: goal, flow, layout, reproduction steps, Phase 1 summary
`1989da2`	11-03 10:01	Phase 2: CTS matrix at 2.2 ns (R0a/R0b/RA/RB_nocl/RB_s4/RB_s20/RC_none/RC_full/RD)
`20e4243`	11-03 10:24	Phase 2 complete: NDR/cluster variants, routing-noise probes, slack decomposition
`759789a`	11-03 10:26	Findings, README results table, complete phase2.sh
`f047339`	11-04 02:05	RD_phys: build useful skew with real delay buffers and re-route
`3cfb844`	11-04 07:34	Useful skew round 2 (RD_phys2, RD_phys2_444) from post-route targets

本報告由 scratchpad 中的產生腳本，從 repo 內的 run 報告直接產生（HTML → Chromium 列印成 PDF）。